Lab 10: Geometry on the GPU

CP479 Computer Graphics -– November 6, 2026

One thread colours one pixel; distances draw a shape; fewer triangles store a shape; then GPU threads measure the error. You fill **seven small tasks** in four files. The hard geometry, camera, lighting, mesh updates and safety checks are given.

Start here: laptop and server

Get your own copy from the course template ↗. Clone it and open a terminal in that folder. You need a C++17 compiler and CMake, as in Lab 09; install instructions and troubleshooting are in `README.md`. Your laptop needs no GPU: the given CPU layer runs the same code.

1. On macOS/Linux, run `make check`. On Windows, run `.\make.bat check`. Initially all seven tasks fail; the programs still compile.
2. Fill a task's TODO blanks, check it, then run its program and view the result.
3. Save, commit and push your files before moving to the server.
4. The TA supplies your account and login address. Compile on the login node; SLURM queues the actual GPU run. Do not compile inside a job.

Terminal

```
# Laptop: after saving your changes
git add student
git commit -m "Complete lab tasks"
git push
# Server: replace the account, address and repository URL
ssh <your-account>@<login-node>
git clone <your-repository-URL> Lab10-GPU-Geometry
cd Lab10-GPU-Geometry
git pull
make -f Makefile.gpu
mkdir -p out
sbatch slurm/hello.sbatch
squeue --me
cat out/lab10-*.out
```

After the first clone, use `git pull`, build again, then submit a job. `PD` means waiting, `R` means running; when the queue line disappears, read the log for errors and the expected panel. Each job has a five-minute limit. The log is `out/lab10-<jobid>.out`, using the job number printed by `sbatch`.

Command convention. Below, `make X ARGS="..."` on macOS/Linux means `.\make.bat X ...` on Windows (omit `ARGS=`). Run from the repository folder. All times are milliseconds (ms).

AI. Keep a prompt log. Understand and be able to explain every submitted line. AI assistants should explain the idea, offer a hint first, explain any code line by line, and ask one short understanding question before moving on. Students write their own measured answers.

Part 0: one thread, one pixel

A kernel is a function run once for each thread. Each thread does one small job. A block groups threads; the grid contains all the blocks. Here each block has $16 \times 16 = 256$ threads. CUDA supplies the indices automatically: do not declare them. The CPU layer supplies the same names on a laptop.

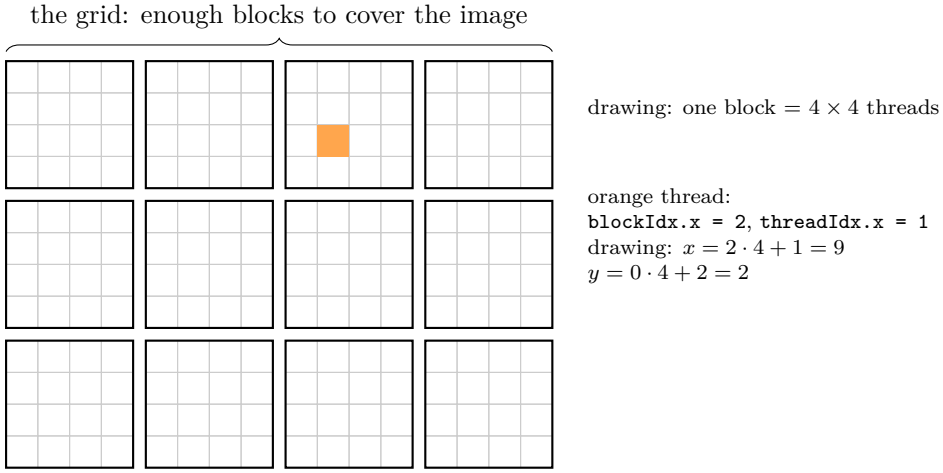


Figure 1: Drawing with blocks of 4: the orange thread is pixel $(9, 2)$. Separately, in the real program with blocks of 16, these indices give $x = 2 \cdot 16 + 1 = 33$, $y = 0 \cdot 16 + 2 = 2$. Each square is one thread.

For a real block width of 16, block 2 and local thread 1 give global x coordinate $2 \cdot 16 + 1 = 33$. Use the same rule for y.

$$x = \texttt{blockIdx.x} * \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

An image 1000 pixels wide needs 63 whole blocks: $63 \cdot 16 = 1008$ threads per row. The extra eight must return without writing. This is a **bounds check**.

One function call does one pixel

`__global__` means a GPU kernel. It returns `void`; each thread stores its answer in the output array. A bare `return;` stops only that thread. The given host code allocates the array, starts the threads and saves the picture.

The array stores row 0, then row 1, and so on. Pixel (x, y) goes at `img[y*w+x]`. For width 4, pixel $(2, 1)$ is cell $1 \cdot 4 + 2 = 6$. Each `Color` is three numbers: red, green and blue, normally from 0 to 1.

Task 0: colour the image

Task 0: `gradient_kernel`

File / output	`student/part0_hello.cu`; `out/hello.png`
Inputs	`Color* img`: writable colour array; `w,h`: image width and height.
Result	Writes one colour at `img[y*w+x]`; no returned value.
Names	`blockIdx`: block number; `blockDim`: block size; `threadIdx`: local position; use both `.x` and `.y`.
Example	Width 4, height 3, pixel $(2, 1)$ gets $(2/3, 1/2, 0.25)$.

What you do: Find this thread's pixel and stop it if it is outside the image.

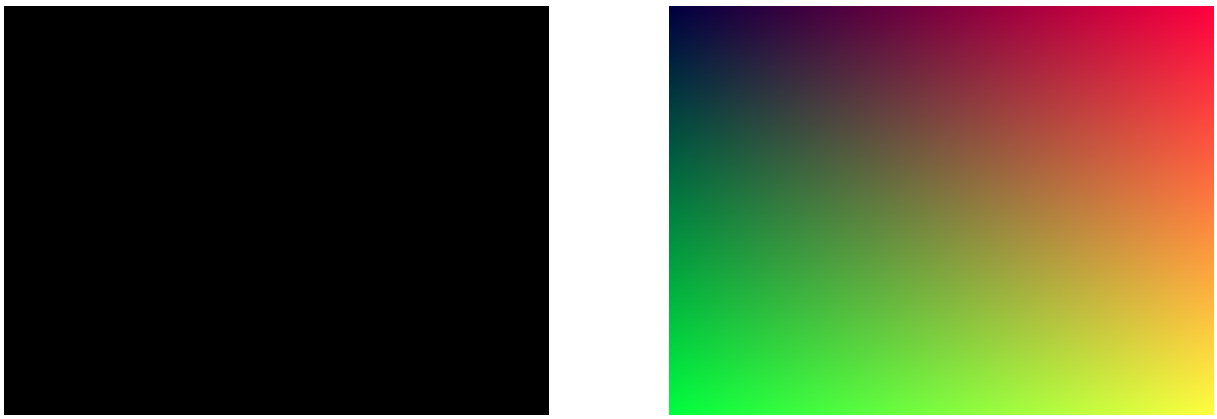
Why: Each valid thread should colour exactly one pixel, producing the gradient below.

How: Fill the numbered TODO blanks in order.

1. Set `x` to block x number times block width plus local x.
2. Set `y` in the same way, using the y indices.
3. Fill the condition: `x >= w || y >= h`; return when it is true.

Run `make check`; look for [0] OK.

The colour assignment is given. It divides x and y by their largest valid values. The `float` conversion keeps fractional colours; integer division would make most pixels dark. Run `make hello` and open `out/hello.png`.



Starter (left); completed Task 0 (right).

Part 0: one answer

How many threads did the program start, and how many pixels does the image have? Why can there be more threads than pixels? Read `Threads / pixels` in the panel.

Check detail. The test uses 37×23 pixels so there are extra threads. It checks the corners and also checks memory beyond the image.

Part I: draw a shape with distances

Instead of triangles, store a function that answers: how far is the surface from this point? A signed distance function (SDF) is positive outside, zero on the surface and negative inside. Your functions return distances. The given renderer makes camera rays and colours the pixels. Leave `HD` in place: these functions run on either CPU or GPU.

For a sphere centred at the origin, the distance is `length(p) - r`. At $(3, 0, 0)$ with radius 1 the answer is 2; at $(0.5, 0, 0)$ it is -0.5.

Task 1a: sphere distance

File / function	`student/part1_sdf.cu`; `sdf_sphere(p,r)`
Inputs	`vec3 p`: point relative to the centre; `float r`: radius.
Result	A `float`: signed distance to the sphere.
Names	`vec3` has fields x, y, z; given `length(p)` measures distance to the origin.
Example	`length(vec3{3,0,0})` is 3; subtract radius 1 to get 2.

What you do: Return the distance to the centre minus the radius.

Why: The ray needs this distance to decide how far it can move.

How: Fill the numbered TODO blanks in order.

1. Replace the return blank with `length(p) - r`.

Run `make check`; look for [1a] OK.

Run `make trace`. There is still only sky: the next task finds ray hits.

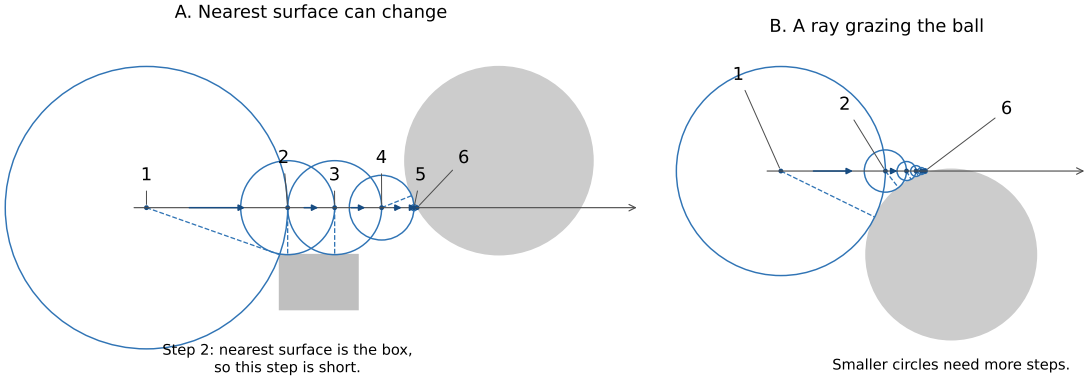


Before ray tracing is finished, there is no visible surface.

Given pieces. `sdf_box(p,half_size)` in `src/core/sdf_given.h` measures distance to an origin-centred box. Its sizes are half the full side lengths. The normal and lighting code are also supplied; you do not need to derive them.

Task 1b: follow a ray

A ray starts at `ro` and moves in unit direction `rd`. At distance t along it, the point is `ro + t * rd`. Ask the SDF for a safe step length, move by that length, and repeat.



For a unit sphere: start $(0, 0, 5)$, direction $(0, 0, -1)$, first distance 4; move to $(0, 0, 1)$, distance 0: hit at t=4 after two samples.

Task 1b: sphere tracing

File / function	`student/part1_sdf.cu`; `sphere_trace(ro,rd,k,steps)`
Inputs	`ro`: origin; `rd`: unit direction; `k`: pass unchanged; `steps`: pointer to the sample count.
Result	Returns hit distance t, or -1 for a miss; writes `*steps`.
Names	Given `current_scene(p,k)` selects the ball until 1d is done; `HIT_EPS=0.001`, `MAX_DIST=20`, `MAX_STEPS=256`.

What you do: Fill four expressions in the given ray loop.

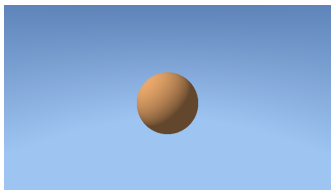
Why: The loop finds the surface for each pixel so the ball becomes visible.

How: Fill the numbered TODO blanks in order.

1. Fill `p` with `ro + t * rd`.
2. Fill `distance` with `current_scene(p,k)`.
3. Fill the hit condition with `distance < HIT_EPS`.
4. Advance t by `distance`.

Run `make check`; look for [1b] OK.

Run `make trace`. You now get a lit ball. Given `sdf_normal` in `src/core/sdf_given.h` samples a little forward and backward on each axis, then normalizes the differences. It tells the given light which way the surface faces.



Task 1c: combine shapes

Sample both shapes at the same point, giving distances a and b. The smaller distance keeps points inside either shape: **union** uses `min(a,b)`. To cut B out of A, flip B's sign and use `max(a,-b)`. For example, a=−1, b=−2 gives subtraction distance 2: that point is removed.

Task 1c: union and subtraction

File / functions	`student/part1_sdf.cu`; `op_union(a,b)`, `op_subtract(a,b)`
Inputs	`float a,b`: signed distances to A and B at the same point.
Result	One `float`: distance field for the combined shape.
Names	Given `min(a,b)` chooses smaller; `max(a,b)` chooses larger.
Examples	Union of $-1, 2$ is -1; subtraction of $-1, -2$ is 2.

What you do: Fill one return expression in each function.

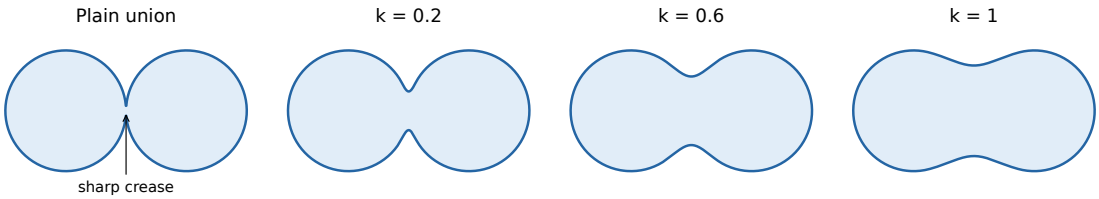
Why: These operations let the next task cut a box and join it to other shapes.

How: Fill the numbered TODO blanks in order.

1. In `op_union`, return `min(a,b)`.
2. In `op_subtract`, return `max(a,-b)`.

Run `make check`; look for [1c] OK.

Given smooth union. `op_smooth_union(a,b,k)` in the same file rounds the join. A larger k makes a wider blend; k=0 gives a hard union. Its complete code is supplied. Pass k through without changing it.



This task does not change the ball picture. It prepares the operations for the full scene.

Task 1d: build the scene

Use the functions you already have: cut a unit sphere out of a box, join a smaller ball above it, and add a floor. To move a sphere's centre to c, sample it with `p - c`.

Task 1d: scene distance

File / function	`student/part1_sdf.cu`; `scene_sdf(p,k)`
Inputs	`vec3 p`: world point; `float k`: blend width.
Result	One signed field value for the whole scene.
Names	Given `sdf_box`, `op_smooth_union`; your sphere, union and subtraction; `p.y + 0.8f` is floor distance.
Example	At the top ball centre $(0, 1.15, 0)$, the scene value is -0.5.

What you do: Fill the three expressions that assemble the given pieces.

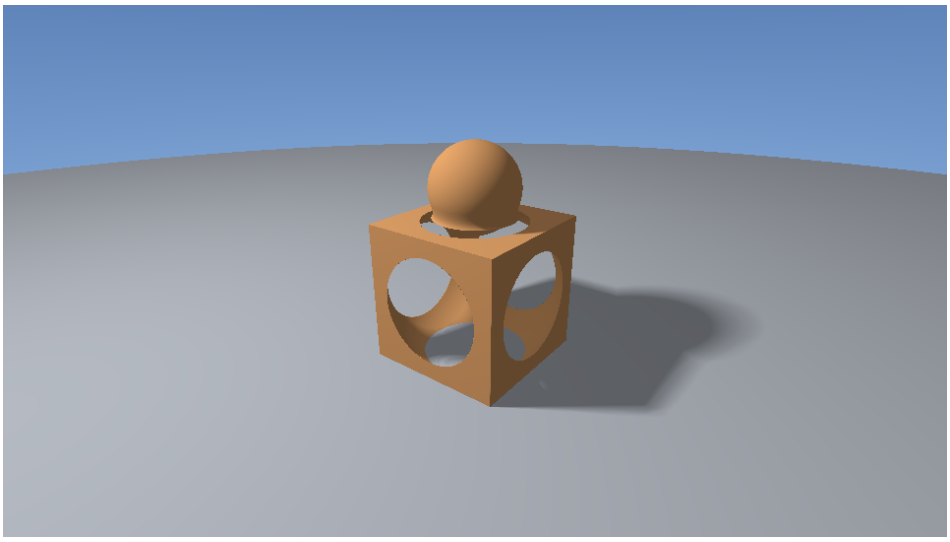
Why: The renderer will switch from the ball to the scene below when this task is complete.

How: Fill the numbered TODO blanks in order.

1. Set `holes` to `op_subtract(sdf_box(p,{0.8f,0.8f,0.8f}), sdf_sphere(p,1))`.
2. Set `top` to `sdf_sphere(p - vec3{0,1.15f,0},0.5f)`.
3. Return `op_union(op_smooth_union(holes,top,k),p.y + 0.8f)`.

Run `make check`; look for [1d] OK.

Run `make trace` and open `out/trace.png`.

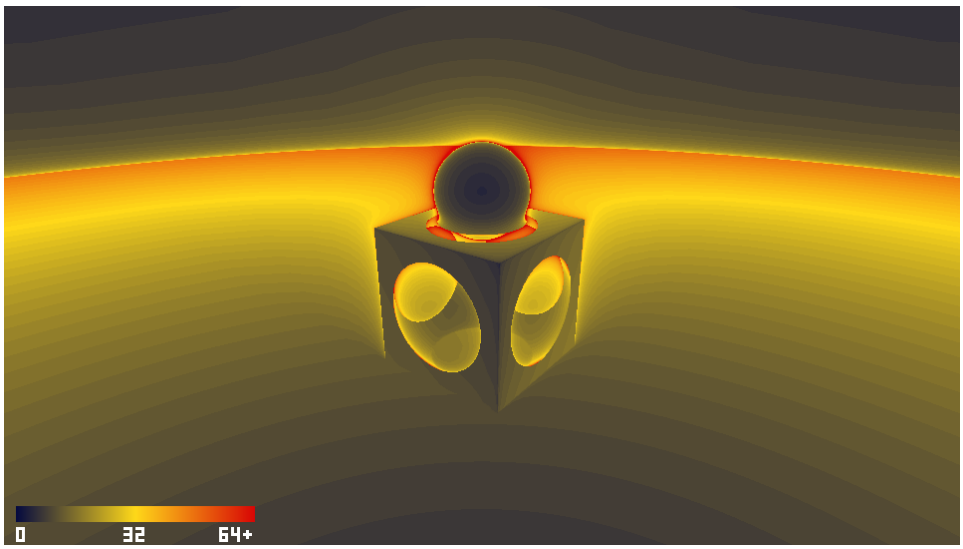


Completed scene, using the supplied light and normal.

The field near a smooth join is a distance estimate. Smaller safe steps can take more iterations. All ray steps, normal samples and shadows use the same selected scene.

Part I: run, look and answer

On your laptop run `make trace ARGS="--mode steps"`; save that picture before another trace run overwrites it. The panel reports the number of steps and the fixed colour scale.



Dark: few steps; yellow: 32; red: 64 or more. The loop can still take up to 256.

For the timing question, push your changes, pull and build on the login node as on page 1. Then submit:

Terminal

```
sbatch --export=ALL,PROGRAM=trace,ARGS="--compare" slurm/run.sbatch
```

Wait until it leaves the queue; read its `out/lab10-<jobid>.out` log. Use `CPU single ms` for CPU time and `Render ms` for GPU time. Check that the backend names the GPU. The panel also prints `CPU single / GPU`.

Part I: two answers and one short play observation

1. Run trace with `--compare` on the server. Write the CPU time, the GPU time and the speedup (speedup = CPU time ÷ GPU time).
2. Look at `--mode steps`. Which parts are bright (many steps)? Why do rays that pass close to a surface need more steps?
3. Try `--k 0` and `--k 1`. What changes in the picture?

Play with it: Part I

Run `make trace ARGS="--k 0"`.
Then run `make trace ARGS="--k 1"`. Save each `out/trace.png` under a different name and compare the ball-to-box join.

Keep the ordinary shaded picture, the steps picture and the two k pictures for your report.

Part II: fewer triangles, same shape

A mesh stores vertices joined into triangles. It is quick to draw, but detailed meshes can be large. Our input, `data/part.obj`, has 36,248 triangles and many unnecessary triangles on flat sides. It is a machine part with a hole, groove and bump, made from another SDF; it is not the scene from Part I.

An **edge collapse** merges two endpoints into one point. Two triangles disappear. Repeat using the least damaging safe collapse and the mesh gets smaller.

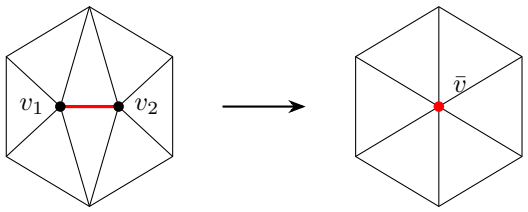


Figure 2: An edge collapse: 1 vertex, 3 edges and 2 triangles disappear.

The given machinery finds each triangle's plane, stores the plane scores, sorts the candidate edges, updates neighbours and refuses unsafe collapses. You only choose the cheapest of three possible replacement points.

A score you can use without matrix algebra

A `Quadric` stores the sum of squared distances to nearby original planes. The given `evaluate(Q,position)` returns that score; lower is better. Adding two quadrics combines their planes. Treat the storage as a black box.

Given plane helper. `triangle_plane` in `src/core/quadric.h` crosses two triangle edges and normalizes the result. The given code converts that plane into a score; you do not need to write either operation.

For the plane z=0, a point at $(0, 0, 2)$ scores $2^2 = 4$. For the three planes x=0, y=0 and z=0, the score is $x^2 + y^2 + z^2$.

Task 2a: choose the cheapest point

Task 2a: collapse cost

File / function	`student/part2_qem.cu`; `collapse_cost(Q,v1,v2,best)`
Inputs	`const Quadric& Q`: combined plane score; `v1,v2`: endpoints; `vec3* best`: writable chosen position.
Result	Writes `*best`; returns its `double` cost.
Names	Given `midpoint=(v1+v2)*0.5f`; `position` is the next candidate; `evaluate(Q,position)` scores it.
Example	Ends $(-1,0,0),(1,0,0)$ cost 1 each for the three origin planes; midpoint $(0,0,0)$ costs 0 and wins.

What you do: Compare the candidate cost and return the winning cost.

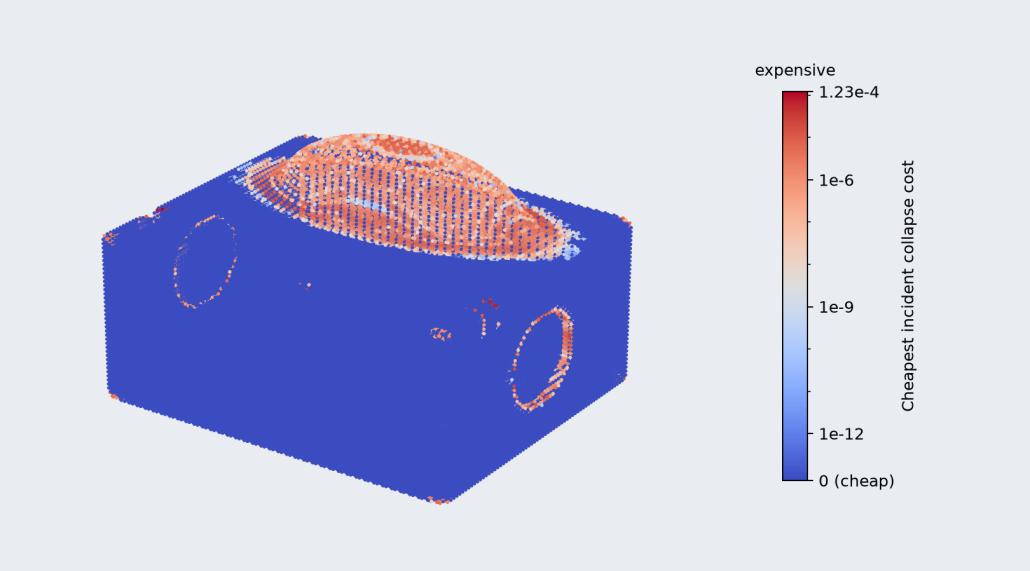
Why: Choosing the least expensive point removes flat detail while keeping important edges.

How: Fill the numbered TODO blanks in order.

1. In the given loop, fill the condition with `evaluate(Q,position) < evaluate(Q,*best)`.
2. Return `evaluate(Q,*best)` after the loop.

Run `make check`; look for [2a] OK.

The midpoint, loop and storage through `*best` are already given. Only the two endpoints and their midpoint are candidates. Any cheapest tie is fine.

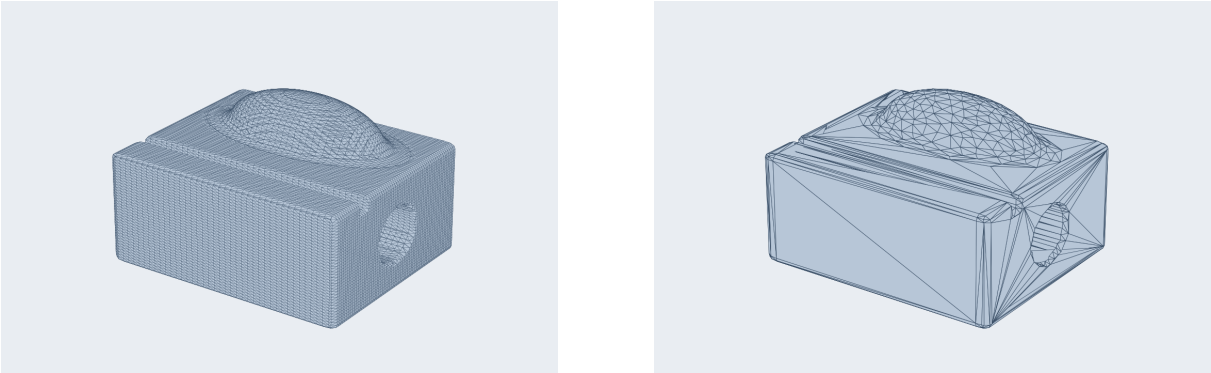


Supplied explanatory rendering: blue vertices have cheap incident collapses; red vertices have expensive ones. This is a cost picture, not output from simplify.

Flat regions can move within a plane without cost. At a corner or hole rim, several planes constrain the point, so more triangles tend to survive. Now run `make simplify ARGS="--target 1000"`.

Part II: inspect the mesh and counts

Open `data/part.obj` and `out/simplified.obj` in a 3D viewer such as 3dviewer.net ↗ or Blender. Turn on triangle edges and rotate the part. The images here are supplied renderings of OBJ files; your viewer camera will differ.



Input: 36,248 triangles (left); baseline: 1000 triangles (right).

The panel prints V (vertices), E (edges), F (faces) and $\chi = V - E + F$. For this input $\chi = 0$. A safe collapse removes one vertex, three edges and two faces: the change is $-1 + 3 - 2 = 0$. Equal χ alone does not guarantee a good shape; use the judge verdict too.

Part II: two answers from the same run

1. Run `make simplify ARGS="--target 1000"` and open the result. Where are many triangles left, and where only a few? Why?
2. Write V, E, F and χ before and after. Did χ change?

Keep a screenshot of the 1000-triangle mesh with its edges visible. Before 3a is finished, the panel says `Part III not done yet`; that is expected. The independent judge can still measure the saved mesh.

Play with it: Part II (not graded)

Try target 3000; try target 500. Save a copy of each OBJ before the next run overwrites it. Rotate the meshes and look at the hole and bump. Restore target 1000 for Part III.

Part III: measure the error

Sample points on A; for each point, find its nearest triangle on B. The largest distance estimates the worst error from A to B. Swap A and B and repeat; take the larger maximum. This is a sampled two-sided **Hausdorff distance**. Samples can miss the true worst point. The panel also reports a percentage of the original bounding-box diagonal so units do not matter.

One point is one thread; each thread searches all destination triangles. The shared maximum and sum need atomic updates, which safely combine simultaneous writes. The distance helper, loop and atomic helper implementations are given.

Task 3a: distance kernel

File / function	`student/part3_metric.cu`; `distance_kernel`
Inputs	`points,n_points`: samples; `tri_verts,n_tris`: target triangles; `out_max,out_sum`: shared outputs, initially zero.
Result	No return value; atomically updates the largest distance and sum.
Names	CUDA x indices as in Task 0; `best_dist=1e30f` starts huge; given `point_triangle_distance` and `atomic_max_float`, `atomic_add_float`.
Example	Two points 1 and 2 units above a triangle give max 2 and sum 3.

What you do: Fill four blanks to choose the nearest distance for each point and combine the results.

Why: This measures how closely the smaller mesh matches the original.

How: Fill the numbered TODO blanks in order.

1. Set i to `blockIdx.x * blockDim.x + threadIdx.x`; the bounds check is given.
2. Inside the triangle loop, keep `min(best_dist,distance)`.
3. Replace the first final placeholder statement with:
   `atomic_max_float(out_max,best_dist);`
4. Replace the second with:
   `atomic_add_float(out_sum,best_dist);`

Run `make check`; look for [3a] OK.

Run `make metric ARGS="data/part.obj out/simplified.obj"` after making the 1000-triangle mesh. Read `A -> B kernel ms` and `B -> A kernel ms`.

Part III: one answer

Which of the two directions took longer? Why? Count the work: points × destination triangles.

Play with it: Part III (not graded)

Swap the two OBJ filenames. Compare the two directional maxima. Look at where the bump was simplified.

Competition and submission

Goal: submit the fewest triangles that pass the independent judge. It checks a closed surface without tears or branches, the same $\chi = 0$, and a sampled two-sided Hausdorff error at most **0.5%** of the original bounding-box diagonal. Among valid meshes, fewer triangles wins; a tie uses the smaller mean error.

First run `make simplify ARGS="--target 1000"`.

Then run `make judge ARGS="out/simplified.obj"`. The unchanged three-candidate baseline is fine and counts as complete. Submit your own generated mesh and its judge line. The mesh must come from your code and `data/part.obj`; do not edit the OBJ by hand or use another decimator.

Optional reading: `extension.pdf` contains the eikonal check, topology theory, unsafe runs, optimal solve, calibration, box SDF details, 4K timing and competition ideas; none is graded.

Commands to keep handy

Command	Purpose
`make check`	Check all seven tasks.
`make trace ARGS="--mode steps"`	Show tracing work.
`make trace ARGS="--k 1"`	Change blend width (any number).
`--compare` (server trace job)	CPU/GPU timing; full sbatch command on page 8.
`make simplify ARGS="--target 1000"`	Choose the target triangle count.
`make metric ARGS="data/part.obj out/simplified.obj"`	Measure both directions.
`make judge ARGS="out/simplified.obj"`	Get the independent verdict.

Submit

- Four files in `student/`: Parts 0, 1, 2 and 3.
- Pictures: hello, shaded trace, steps, k=0, k=1, and the 1000-triangle mesh with edges visible.
- The short answers on pages 3, 8, 11 and 12, using your own runs.
- Your competition mesh, named `<student_id>.obj`, its generating command and judge line.
- AI log: prompt, model, what you needed/already knew, and how you checked each answer. If you used no AI, say so.

Background: Hart (1996), Sphere Tracing; Garland and Heckbert (1997), Surface Simplification Using Quadric Error Metrics; Cignoni et al. (1998), Metro; NVIDIA CUDA Programming Guide.